

Dentist Clinic Simulation

1.0

Generated by Doxygen 1.15.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 patient Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 appointment	5
3.1.2.2 arrivalOrder	6
3.1.2.3 dentistCallSem	6
3.1.2.4 id	6
3.1.2.5 leftDoctorFull	6
3.1.2.6 leftNoChair	6
3.1.2.7 name	6
3.1.2.8 noShow	7
3.1.2.9 replySem	7
3.1.2.10 requestPending	7
3.1.2.11 treated	7
3.1.2.12 treatmentDoneSem	7
3.1.2.13 waiting	7
3.1.2.14 willArrive	7
4 File Documentation	9
4.1 dentist.c File Reference	9
4.1.1 Detailed Description	10
4.1.2 Function Documentation	12
4.1.2.1 dentist()	12
4.1.2.2 destroyPatient()	14
4.1.2.3 findNextPatient()	15
4.1.2.4 findNextRequest()	15
4.1.2.5 initializePatient()	16
4.1.2.6 main()	17
4.1.2.7 patient()	21
4.1.2.8 printPatientTable()	24
4.1.2.9 printStatistics()	25
4.1.2.10 receptionist()	26
4.1.2.11 showProjectInfo()	28
4.1.3 Variable Documentation	29
4.1.3.1 activeAppointments	29
4.1.3.2 appointmentSlots	29

4.1.3.3 arrivalCounter	29
4.1.3.4 clinicClosed	29
4.1.3.5 confirmedAppointments	30
4.1.3.6 doctorCapacity	30
4.1.3.7 mutex	30
4.1.3.8 patientReady	30
4.1.3.9 patients	30
4.1.3.10 patientThreads	30
4.1.3.11 requestReady	31
4.1.3.12 totalPatients	31
4.1.3.13 treatedPatients	31
4.1.3.14 waitingPatients	31
4.1.3.15 waitingRoomChairs	31
4.2 dentist.c	32
Index	45

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

patient	Stores all information about a patient	5
-------------------------	--	-------------------

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

dentist.c	Dentist Clinic Simulation - Operating System Project	9
---------------------------	--	---

Chapter 3

Class Documentation

3.1 patient Struct Reference

Stores all information about a patient.

Public Attributes

- int [id](#)
- char [name](#) [64]
- int [appointment](#)
- int [willArrive](#)
- int [arrivalOrder](#)
- int [waiting](#)
- int [treated](#)
- int [noShow](#)
- int [leftNoChair](#)
- int [leftDoctorFull](#)
- int [requestPending](#)
- sem_t [replySem](#)
- sem_t [dentistCallSem](#)
- sem_t [treatmentDoneSem](#)

3.1.1 Detailed Description

Stores all information about a patient.

This structure stores appointment information, waiting room information, treatment status and private semaphores.

Definition at line [64](#) of file [dentist.c](#).

3.1.2 Member Data Documentation

3.1.2.1 appointment

```
int patient::appointment
```

1 = confirmed, 0 = not confirmed

Definition at line [69](#) of file [dentist.c](#).

3.1.2.2 arrivalOrder

```
int patient::arrivalOrder
```

FIFO / FCFS arrival order

Definition at line 72 of file [dentist.c](#).

3.1.2.3 dentistCallSem

```
sem_t patient::dentistCallSem
```

Dentist call semaphore

Definition at line 83 of file [dentist.c](#).

3.1.2.4 id

```
int patient::id
```

Patient ID

Definition at line 66 of file [dentist.c](#).

3.1.2.5 leftDoctorFull

```
int patient::leftDoctorFull
```

1 = left because doctor was full

Definition at line 78 of file [dentist.c](#).

3.1.2.6 leftNoChair

```
int patient::leftNoChair
```

1 = left because room was full

Definition at line 77 of file [dentist.c](#).

3.1.2.7 name

```
char patient::name[64]
```

Patient name

Definition at line 67 of file [dentist.c](#).

3.1.2.8 noShow

```
int patient::noShow
```

1 = no-show

Definition at line 76 of file [dentist.c](#).

3.1.2.9 replySem

```
sem_t patient::replySem
```

Receptionist reply semaphore

Definition at line 82 of file [dentist.c](#).

3.1.2.10 requestPending

```
int patient::requestPending
```

1 = appointment request pending

Definition at line 80 of file [dentist.c](#).

3.1.2.11 treated

```
int patient::treated
```

1 = treated

Definition at line 75 of file [dentist.c](#).

3.1.2.12 treatmentDoneSem

```
sem_t patient::treatmentDoneSem
```

Treatment completion semaphore

Definition at line 84 of file [dentist.c](#).

3.1.2.13 waiting

```
int patient::waiting
```

1 = waiting, 0 = not waiting

Definition at line 73 of file [dentist.c](#).

3.1.2.14 willArrive

```
int patient::willArrive
```

1 = will arrive, 0 = no-show

Definition at line 70 of file [dentist.c](#).

The documentation for this struct was generated from the following file:

- [dentist.c](#)

Chapter 4

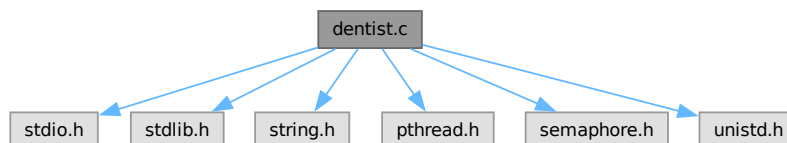
File Documentation

4.1 dentist.c File Reference

Dentist Clinic Simulation - Operating System Project.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

Include dependency graph for dentist.c:



Classes

- struct [patient](#)
Stores all information about a patient.

Functions

- void [initializePatient](#) (int i)
Initializes a patient structure.
- void [destroyPatient](#) (int i)
Destroys all semaphores of a patient.
- int [findNextPatient](#) ()
Finds the next patient using FIFO / FCFS.

- int `findNextRequest` ()
Finds the next appointment request.
- void * `receptionist` (void *arg)
Receptionist thread function.
- void * `dentist` (void *arg)
Dentist thread function.
- void * `patient` (void *arg)
Patient thread function.
- void `printPatientTable` ()
Displays the final patient table.
- void `printStatistics` ()
Displays final clinic statistics.
- void `showProjectInfo` ()
Displays the contents of project_info.txt.
- int `main` ()
Main entry point of the program.

Variables

- int `totalPatients`
- int `waitingRoomChairs`
- int `doctorCapacity`
- int `appointmentSlots`
- int `waitingPatients` = 0
- int `treatedPatients` = 0
- int `confirmedAppointments` = 0
- int `activeAppointments` = 0
- int `arrivalCounter` = 0
- int `clinicClosed` = 0
- struct `patient` * `patients`
Pointer to dynamically allocated patient array.
- pthread_t * `patientThreads`
Pointer to dynamically allocated patient thread array.
- pthread_mutex_t `mutex`
Mutex used to protect shared data.
- sem_t `requestReady`
Semaphore used to notify the receptionist when a patient requests an appointment.
- sem_t `patientReady`
Semaphore used to notify the dentist when a patient enters the waiting room.

4.1.1 Detailed Description

Dentist Clinic Simulation - Operating System Project.

Author

shanjeda alam

Version

1.0

Date

2026

This program simulates a dentist clinic using different Operating System concepts.

Main OS Concepts:

1. POSIX Threads
2. Mutex
3. Semaphores
4. FIFO / FCFS Scheduling
5. Critical Section
6. Synchronization
7. Structure
8. Pointer
9. Dynamic Memory Allocation

Main Features:

- Patient management
- Appointment management
- Waiting room management
- Doctor treatment management
- No-show patient handling
- FIFO / FCFS scheduling
- Thread synchronization
- Mutex protected critical sections
- Semaphore based communication

User Input:

- Number of patients
- Number of waiting room chairs
- Doctor capacity
- Number of appointment slots
- Patient names
- Patient arrival / no-show status

Special Rule: If a confirmed appointment patient becomes a no-show, the appointment slot is released and another waiting patient can receive treatment.

Definition in file [dentist.c](#).

4.1.2 Function Documentation

4.1.2.1 dentist()

```
void * dentist (
    void * arg)
```

Dentist thread function.

Dentist thread.

Selects patients using FIFO / FCFS and treats them.

Parameters

<i>arg</i>	Thread argument.
------------	------------------

Returns

NULL when the thread finishes.

The dentist selects waiting patients according to FIFO/FCFS. Each patient receives treatment for a simulated period.

Parameters

<i>arg</i>	Thread argument.
------------	------------------

Returns

NULL when dentist exits.

Definition at line 488 of file [dentist.c](#).

```
00489 {
00490     (void) arg;
00491
00492     printf("\n[Dr. Smith] Clinic is open.\n");
00493
00494     while (1)
00495     {
00496         /* Wait for patient */
00497         sem_wait(&patientReady);
00498
00499         pthread_mutex_lock(&mutex);
00500
00501         /* Close clinic when no patient remains */
00502         if (clinicClosed == 1 &&
00503             waitingPatients == 0)
00504         {
00505             pthread_mutex_unlock(&mutex);
00506             break;
00507         }
00508
00509         /* Doctor has treatment capacity */
00510         if (waitingPatients > 0 &&
00511             treatedPatients < doctorCapacity)
00512         {
00513             int id = findNextPatient();
00514
00515             if (id != -1)
00516             {
00517                 patients[id].waiting = 0;
```

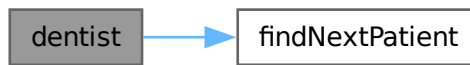


```

00518         waitingPatients--;
00519
00520         printf("\n[Dr. Smith] Calling %s (ID %d)\n",
00521             patients[id].name,
00522             patients[id].id);
00523
00524         printf("[Dr. Smith] FCFS Order: #%d\n",
00525             patients[id].arrivalOrder);
00526
00527         printf("[Dr. Smith] Waiting room: %d/%d\n",
00528             waitingPatients,
00529             waitingRoomChairs);
00530
00531         pthread_mutex_unlock(&mutex);
00532
00533         /* Call exact patient */
00534         sem_post(&patients[id].dentistCallSem);
00535
00536         printf("[Dr. Smith] Treating %s...\n",
00537             patients[id].name);
00538
00539         /* Treatment simulation */
00540         sleep(2);
00541
00542         printf("[Dr. Smith] Treatment finished for %s.\n",
00543             patients[id].name);
00544
00545         pthread_mutex_lock(&mutex);
00546
00547         treatedPatients++;
00548         patients[id].treated = 1;
00549
00550         /* Release appointment slot */
00551         if (patients[id].appointment == 1 &&
00552             activeAppointments > 0)
00553         {
00554             activeAppointments--;
00555         }
00556
00557         pthread_mutex_unlock(&mutex);
00558
00559         /* Notify patient */
00560         sem_post(&patients[id].treatmentDoneSem);
00561     }
00562     else
00563     {
00564         pthread_mutex_unlock(&mutex);
00565     }
00566 }
00567
00568 /* Doctor capacity reached */
00569 else if (waitingPatients > 0 &&
00570         treatedPatients >= doctorCapacity)
00571 {
00572     printf("\n[Dr. Smith] Daily treatment limit reached.\n");
00573     printf("[Dr. Smith] Remaining patients must leave.\n");
00574
00575     for (int i = 0; i < totalPatients; i++)
00576     {
00577         if (patients[i].waiting == 1)
00578         {
00579             patients[i].waiting = 0;
00580             patients[i].leftDoctorFull = 1;
00581
00582             waitingPatients--;
00583
00584             printf("[Dr. Smith] %s (ID %d) - "
00585                 "Left because doctor is full.\n",
00586                 patients[i].name,
00587                 patients[i].id);
00588
00589             sem_post(&patients[i].dentistCallSem);
00590         }
00591     }
00592     pthread_mutex_unlock(&mutex);
00593 }
00594 else
00595 {
00596     pthread_mutex_unlock(&mutex);
00597 }
00598 }
00599 }
00600
00601 printf("\n[Dr. Smith] Closing.\n");
00602
00603 return NULL;
00604 }

```

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.2 destroyPatient()

```
void destroyPatient (  
    int i)
```

Destroys all semaphores of a patient.

Destroys patient semaphores.

Parameters

<i>i</i>	Index of the patient.
----------	-----------------------

Definition at line 294 of file [dentist.c](#).

```
00295 {  
00296     sem_destroy(&patients[i].replySem);  
00297     sem_destroy(&patients[i].dentistCallSem);  
00298     sem_destroy(&patients[i].treatmentDoneSem);  
00299 }
```

Here is the caller graph for this function:



4.1.2.3 findNextPatient()

```
int findNextPatient ()
```

Finds the next patient using FIFO / FCFS.

Finds the next waiting patient using FIFO / FCFS.

Returns

Patient index if available, otherwise -1.

The patient with the smallest arrival order is selected.

Returns

Patient index or -1 if no patient is waiting.

Definition at line 313 of file [dentist.c](#).

```
00314 {  
00315     int selected = -1;  
00316     int smallestOrder = 999999;  
00317  
00318     for (int i = 0; i < totalPatients; i++)  
00319     {  
00320         if (patients[i].waiting == 1 &&  
00321             patients[i].arrivalOrder < smallestOrder)  
00322         {  
00323             smallestOrder = patients[i].arrivalOrder;  
00324             selected = i;  
00325         }  
00326     }  
00327  
00328     return selected;  
00329 }
```

Here is the caller graph for this function:



4.1.2.4 findNextRequest()

```
int findNextRequest ()
```

Finds the next appointment request.

Finds the next patient requesting an appointment.

Returns

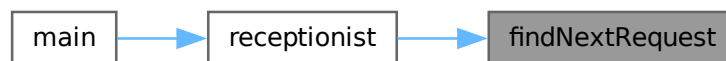
Patient index if request exists, otherwise -1.

Patient index or -1 if no request exists.

Definition at line 341 of file [dentist.c](#).

```
00342 {
00343     for (int i = 0; i < totalPatients; i++)
00344     {
00345         if (patients[i].requestPending == 1)
00346         {
00347             return i;
00348         }
00349     }
00350     return -1;
00351 }
00352 }
```

Here is the caller graph for this function:

**4.1.2.5 initializePatient()**

```
void initializePatient (
    int i)
```

Initializes a patient structure.

Initializes one patient.

Parameters

<i>i</i>	Index of the patient.
----------	-----------------------

All patient status values are set to their default values. Private semaphores are also initialized here.

Parameters

<i>i</i>	Index of the patient in the patient array.
----------	--

Definition at line 262 of file [dentist.c](#).

```
00263 {
00264     patients[i].id = i + 1;
00265     patients[i].appointment = 0;
00266     patients[i].willArrive = 1;
00267     patients[i].arrivalOrder = 0;
00268 }
00269 }
```

```

00270     patients[i].waiting = 0;
00271
00272     patients[i].treated = 0;
00273     patients[i].noShow = 0;
00274     patients[i].leftNoChair = 0;
00275     patients[i].leftDoctorFull = 0;
00276
00277     patients[i].requestPending = 0;
00278
00279     sem_init(&patients[i].replySem, 0, 0);
00280     sem_init(&patients[i].dentistCallSem, 0, 0);
00281     sem_init(&patients[i].treatmentDoneSem, 0, 0);
00282 }

```

Here is the caller graph for this function:



4.1.2.6 main()

```
int main ()
```

Main entry point of the program.

Takes clinic settings and patient information from the user, initializes threads, mutexes and semaphores, starts the simulation and finally cleans up all resources.

Returns

0 if the program finishes successfully.

Definition at line 965 of file [dentist.c](#).

```

00966 {
00967     int i;
00968
00969     printf("\n");
00970     printf("=====\n");
00971     printf("          DENTIST CLINIC SIMULATION\n");
00972     printf("          OS PROJECT\n");
00973     printf("=====\n");
00974
00975
00976     /* -----
00977        SHOW PROJECT INFORMATION
00978        ----- */
00979
00980     showProjectInfo();
00981
00982
00983     /* -----
00984        USER INPUT
00985        ----- */
00986
00987     printf("\nEnter number of patients: ");
00988     scanf("%d", &totalPatients);
00989
00990     if (totalPatients <= 0)
00991     {
00992         printf("Invalid number of patients.\n");

```

```

00993         return 1;
00994     }
00995
00996     printf("Enter number of waiting room chairs: ");
00997     scanf("%d", &waitingRoomChairs);
00998
00999     if (waitingRoomChairs <= 0)
01000     {
01001         printf("Invalid number of chairs.\n");
01002         return 1;
01003     }
01004
01005     printf("Enter doctor's maximum capacity: ");
01006     scanf("%d", &doctorCapacity);
01007
01008     if (doctorCapacity <= 0)
01009     {
01010         printf("Invalid doctor capacity.\n");
01011         return 1;
01012     }
01013
01014     printf("Enter number of appointment slots: ");
01015     scanf("%d", &appointmentSlots);
01016
01017     if (appointmentSlots <= 0 ||
01018         appointmentSlots > totalPatients)
01019     {
01020         printf("Invalid appointment slot number.\n");
01021         return 1;
01022     }
01023
01024     /* -----
01025     MEMORY ALLOCATION
01026     ----- */
01027
01028     patients =
01029         malloc(totalPatients * sizeof(struct patient));
01030
01031     patientThreads =
01032         malloc(totalPatients * sizeof(pthread_t));
01033
01034     if (patients == NULL ||
01035         patientThreads == NULL)
01036     {
01037         printf("Memory allocation failed.\n");
01038
01039         free(patients);
01040         free(patientThreads);
01041
01042         return 1;
01043     }
01044
01045     /* -----
01046     PATIENT INPUT
01047     ----- */
01048
01049     printf("\n-----\n");
01050     printf("Enter patient information\n");
01051     printf("1 = Patient will come\n");
01052     printf("0 = Patient will not come (No-show)\n");
01053     printf("-----\n");
01054
01055     for (i = 0; i < totalPatients; i++)
01056     {
01057         initializePatient(i);
01058
01059         printf("\nPatient %d name: ",
01060             i + 1);
01061
01062         scanf("%63s",
01063             patients[i].name);
01064
01065         printf("Will %s come? (1=Yes, 0=No): ",
01066             patients[i].name);
01067
01068         scanf("%d",
01069             &patients[i].willArrive);
01070     }
01071
01072     /* -----
01073     CLINIC SETTINGS

```

```

01080     ----- */
01081
01082     printf("\n=====\\n");
01083     printf("                CLINIC SETTINGS\\n");
01084     printf("=====\\n");
01085
01086     printf("Total patients      : %d\\n",
01087            totalPatients);
01088
01089     printf("Waiting room chairs : %d\\n",
01090            waitingRoomChairs);
01091
01092     printf("Doctor capacity      : %d\\n",
01093            doctorCapacity);
01094
01095     printf("Appointment slots   : %d\\n",
01096            appointmentSlots);
01097
01098     printf("Scheduling          : FIFO / FCFS\\n");
01099
01100     printf("=====\\n");
01101
01102
01103     /* -----
01104     INITIALIZE SYNCHRONIZATION
01105     ----- */
01106
01107     pthread_mutex_init(&mutex, NULL);
01108
01109     sem_init(&requestReady, 0, 0);
01110     sem_init(&patientReady, 0, 0);
01111
01112
01113     /* -----
01114     CREATE DENTIST THREAD
01115     ----- */
01116
01117     pthread_t dentistThread;
01118
01119     pthread_create(&dentistThread,
01120                   NULL,
01121                   dentist,
01122                   NULL);
01123
01124
01125     /* -----
01126     CREATE RECEPTIONIST THREAD
01127     ----- */
01128
01129     pthread_t receptionistThread;
01130
01131     pthread_create(&receptionistThread,
01132                   NULL,
01133                   receptionist,
01134                   NULL);
01135
01136
01137     printf("\n=====\\n");
01138     printf("                SIMULATION STARTED\\n");
01139     printf("=====\\n");
01140
01141
01142     /* -----
01143     CREATE PATIENT THREADS
01144     ----- */
01145
01146     for (i = 0; i < totalPatients; i++)
01147     {
01148         pthread_create(&patientThreads[i],
01149                       NULL,
01150                       patient,
01151                       &patients[i]);
01152
01153         /*
01154          * Small delay between patient arrivals.
01155          */
01156         usleep(300000);
01157     }
01158
01159
01160     /* -----
01161     WAIT FOR PATIENT THREADS
01162     ----- */
01163
01164     for (i = 0; i < totalPatients; i++)
01165     {
01166         pthread_join(patientThreads[i],

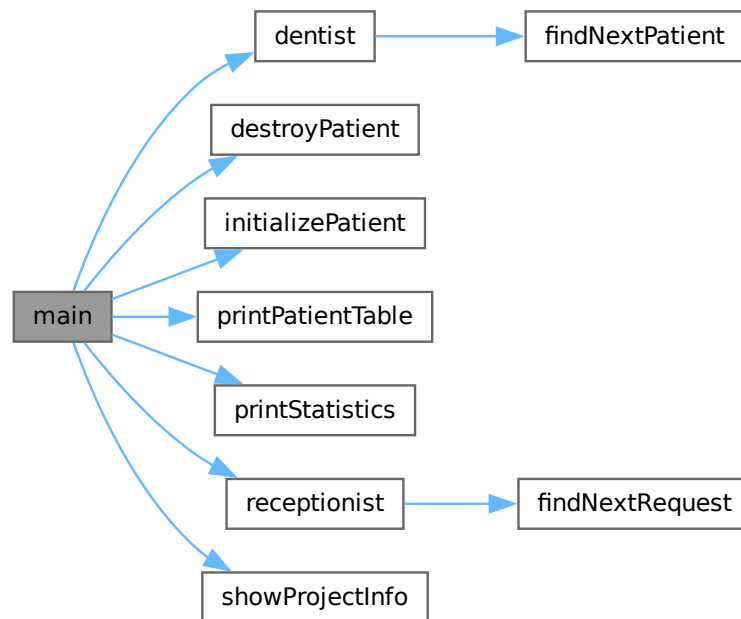
```

```

01167             NULL);
01168     }
01169
01170
01171     /* -----
01172     CLOSE CLINIC
01173     ----- */
01174
01175     pthread_mutex_lock(&mutex);
01176     clinicClosed = 1;
01177
01178     pthread_mutex_unlock(&mutex);
01179
01180
01181     /* Wake dentist and receptionist */
01182     sem_post(&patientReady);
01183     sem_post(&requestReady);
01184
01185
01186     /* Wait for dentist */
01187     pthread_join(dentistThread,
01188                 NULL);
01189
01190
01191     /* Wait for receptionist */
01192     pthread_join(receptionistThread,
01193                 NULL);
01194
01195
01196     /* -----
01197     FINAL RESULT
01198     ----- */
01199
01200
01201     printf("\n===== \n");
01202     printf("                CLINIC CLOSED\n");
01203     printf("===== \n");
01204
01205     printPatientTable();
01206
01207     printStatistics();
01208
01209
01210     /* -----
01211     CLEANUP
01212     ----- */
01213
01214     pthread_mutex_destroy(&mutex);
01215
01216     sem_destroy(&requestReady);
01217     sem_destroy(&patientReady);
01218
01219     for (i = 0; i < totalPatients; i++)
01220     {
01221         destroyPatient(i);
01222     }
01223
01224     free(patients);
01225     free(patientThreads);
01226
01227     printf("\nAll threads terminated successfully.\n");
01228     printf("Program finished.\n\n");
01229
01230     return 0;
01231 }

```


Here is the call graph for this function:



4.1.2.7 patient()

```
void * patient (  
    void * arg)
```

Patient thread function.

Patient thread.

Handles appointment request, arrival, waiting room, no-show and treatment.

Parameters

<i>arg</i>	Pointer to a patient structure.
------------	---------------------------------

Returns

NULL when the thread finishes.

Handles:

- Appointment request

- Appointment confirmation
- No-show
- Waiting room entry
- FIFO order
- Treatment

Parameters

<i>arg</i>	Pointer to struct patient.
------------	----------------------------

Returns

NULL when patient thread finishes.

Definition at line 625 of file [dentist.c](#).

```

00626 {
00627     /*
00628      * Pointer to patient structure.
00629      */
00630     struct patient *p = (struct patient *)arg;
00631
00632
00633     /* -----
00634      REQUEST APPOINTMENT
00635      ----- */
00636
00637     printf("\n[Patient %d | %s] Requesting appointment.\n",
00638           p->id,
00639           p->name);
00640
00641     pthread_mutex_lock(&mutex);
00642
00643     p->requestPending = 1;
00644
00645     pthread_mutex_unlock(&mutex);
00646
00647     /* Notify receptionist */
00648     sem_post(&requestReady);
00649
00650     /* Wait for receptionist */
00651     sem_wait(&p->replySem);
00652
00653     printf("[Patient %d | %s] Appointment: ",
00654           p->id,
00655           p->name);
00656
00657     if (p->appointment == 1)
00658     {
00659         printf("CONFIRMED\n");
00660     }
00661     else
00662     {
00663         printf("NOT CONFIRMED / WALK-IN\n");
00664     }
00665
00666
00667     /* -----
00668      NO-SHOW
00669      ----- */
00670
00671     if (p->willArrive == 0)
00672     {
00673         pthread_mutex_lock(&mutex);
00674
00675         p->noShow = 1;
00676
00677         printf("\n[Patient %d | %s] NO-SHOW.\n",
00678               p->id,
00679               p->name);
00680
00681         /*
00682          * Release confirmed appointment slot.

```

```

00683     */
00684     if (p->appointment == 1)
00685     {
00686         if (activeAppointments > 0)
00687         {
00688             activeAppointments--;
00689         }
00690
00691         printf("[Receptionist] %s's appointment slot "
00692             "has been released.\n",
00693             p->name);
00694
00695         printf("[Receptionist] Active appointments: "
00696             "%d/%d\n",
00697             activeAppointments,
00698             appointmentSlots);
00699     }
00700
00701     pthread_mutex_unlock(&mutex);
00702
00703     return NULL;
00704 }
00705
00706
00707 /* -----
00708 ARRIVE AT CLINIC
00709 ----- */
00710
00711 printf("[Patient %d | %s] Arrived at the clinic.\n",
00712     p->id,
00713     p->name);
00714
00715 pthread_mutex_lock(&mutex);
00716
00717 /* Check doctor capacity */
00718
00719 if (treatedPatients >= doctorCapacity)
00720 {
00721     p->leftDoctorFull = 1;
00722
00723     printf("[Patient %d | %s] Doctor is full. "
00724         "Leaving clinic.\n",
00725         p->id,
00726         p->name);
00727
00728     pthread_mutex_unlock(&mutex);
00729
00730     return NULL;
00731 }
00732
00733
00734 /* -----
00735 ENTER WAITING ROOM
00736 ----- */
00737
00738 if (waitingPatients < waitingRoomChairs)
00739 {
00740     arrivalCounter++;
00741
00742     p->arrivalOrder = arrivalCounter;
00743     p->waiting = 1;
00744
00745     waitingPatients++;
00746
00747     printf("[Patient %d | %s] Sitting in waiting room.\n",
00748         p->id,
00749         p->name);
00750
00751     printf("[Patient %d | %s] Chair %d/%d | "
00752         "FCFS Order #%d\n",
00753         p->id,
00754         p->name,
00755         waitingPatients,
00756         waitingRoomChairs,
00757         p->arrivalOrder);
00758
00759     pthread_mutex_unlock(&mutex);
00760
00761     /* Notify dentist */
00762     sem_post(&patientReady);
00763
00764     /* Wait until dentist calls this patient */
00765     sem_wait(&p->dentistCallSem);
00766
00767     pthread_mutex_lock(&mutex);
00768
00769

```

```

00770         if (p->leftDoctorFull == 1)
00771         {
00772             printf("[Patient %d | %s] Sent home because "
00773                 "doctor is full.\n",
00774                 p->id,
00775                 p->name);
00776
00777             pthread_mutex_unlock(&mutex);
00778
00779             return NULL;
00780         }
00781
00782         pthread_mutex_unlock(&mutex);
00783
00784         printf("[Patient %d | %s] Entering treatment room.\n",
00785             p->id,
00786             p->name);
00787
00788         /* Wait for treatment */
00789         sem_wait(&p->treatmentDoneSem);
00790
00791         printf("[Patient %d | %s] Treatment completed. "
00792             "Leaving clinic.\n",
00793             p->id,
00794             p->name);
00795     }
00796
00797
00798     /* -----
00799     WAITING ROOM FULL
00800     ----- */
00801
00802     else
00803     {
00804         p->leftNoChair = 1;
00805
00806         printf("[Patient %d | %s] Waiting room is FULL "
00807             "(%d/%d). Leaving clinic.\n",
00808             p->id,
00809             p->name,
00810             waitingPatients,
00811             waitingRoomChairs);
00812
00813         pthread_mutex_unlock(&mutex);
00814     }
00815
00816     return NULL;
00817 }

```

4.1.2.8 printPatientTable()

```
void printPatientTable ()
```

Displays the final patient table.

Displays final results of all patients.

Definition at line 827 of file [dentist.c](#).

```

00828 {
00829     printf("\n");
00830
00831     printf("+-----+-----+-----+-----+\n");
00832     printf("| ID   | Name           | Appointment | Result           |\n");
00833     printf("+-----+-----+-----+-----+\n");
00834
00835     for (int i = 0; i < totalPatients; i++)
00836     {
00837         char appointmentText[20];
00838         char resultText[30];
00839
00840         if (patients[i].appointment == 1)
00841         {
00842             strcpy(appointmentText, "Confirmed");
00843         }
00844         else
00845         {
00846             strcpy(appointmentText, "Not confirmed");
00847         }
00848

```

```

00849         if (patients[i].treated == 1)
00850         {
00851             strcpy(resultText, "Treated");
00852         }
00853         else if (patients[i].noShow == 1)
00854         {
00855             strcpy(resultText, "No-show");
00856         }
00857         else if (patients[i].leftNoChair == 1)
00858         {
00859             strcpy(resultText, "Room full");
00860         }
00861         else if (patients[i].leftDoctorFull == 1)
00862         {
00863             strcpy(resultText, "Doctor full");
00864         }
00865         else
00866         {
00867             strcpy(resultText, "Left");
00868         }
00869
00870         printf("| %-3d | %-16s | %-13s | %-16s |\n",
00871             patients[i].id,
00872             patients[i].name,
00873             appointmentText,
00874             resultText);
00875     }
00876
00877     printf("+-----+-----+-----+-----+\n");
00878 }

```

Here is the caller graph for this function:



4.1.2.9 printStatistics()

```
void printStatistics ()
```

Displays final clinic statistics.

Definition at line 888 of file [dentist.c](#).

```

00889 {
00890     int noShowCount = 0;
00891     int roomFullCount = 0;
00892     int doctorFullCount = 0;
00893
00894     for (int i = 0; i < totalPatients; i++)
00895     {
00896         if (patients[i].noShow == 1)
00897         {
00898             noShowCount++;
00899         }
00900
00901         if (patients[i].leftNoChair == 1)
00902         {
00903             roomFullCount++;
00904         }
00905
00906         if (patients[i].leftDoctorFull == 1)
00907         {
00908             doctorFullCount++;
00909         }
00910     }

```

```

00911
00912     printf("\n");
00913
00914     printf("+-----+ \n");
00915     printf(" |                STATISTICS                | \n");
00916     printf("+-----+ \n");
00917
00918     printf(" | Total patients      : %-16d | \n",
00919           totalPatients);
00920
00921     printf(" | Confirmed appointments: %-16d | \n",
00922           confirmedAppointments);
00923
00924     printf(" | Patients treated      : %-16d | \n",
00925           treatedPatients);
00926
00927     printf(" | No-show patients      : %-16d | \n",
00928           noShowCount);
00929
00930     printf(" | Room-full patients    : %-16d | \n",
00931           roomFullCount);
00932
00933     printf(" | Doctor-full patients  : %-16d | \n",
00934           doctorFullCount);
00935
00936     printf(" | Waiting room chairs   : %-16d | \n",
00937           waitingRoomChairs);
00938
00939     printf(" | Doctor capacity       : %-16d | \n",
00940           doctorCapacity);
00941
00942     printf(" | Appointment slots     : %-16d | \n",
00943           appointmentSlots);
00944
00945     printf(" | Scheduling            : %-16s | \n",
00946           "FIFO / FCFS");
00947
00948     printf("+-----+ \n");
00949 }

```

Here is the caller graph for this function:



4.1.2.10 receptionist()

```

void * receptionist (
    void * arg)

```

Receptionist thread function.

Receptionist thread.

Handles appointment requests from patients.

Parameters

<i>arg</i>	Thread argument.
------------	------------------

Returns

NULL when the thread finishes.

The receptionist waits for appointment requests and assigns available appointment slots.

Parameters

<i>arg</i>	Thread argument.
------------	------------------

Returns

NULL when receptionist exits.

Definition at line 407 of file [dentist.c](#).

```

00408 {
00409     (void) arg;
00410
00411     printf("\n[Receptionist] Online.\n");
00412
00413     while (1)
00414     {
00415         /* Wait for appointment request */
00416         sem_wait(&requestReady);
00417
00418         pthread_mutex_lock(&mutex);
00419
00420         /* Check clinic closing */
00421         if (clinicClosed == 1)
00422         {
00423             pthread_mutex_unlock(&mutex);
00424             break;
00425         }
00426
00427         /* Find request */
00428         int id = findNextRequest();
00429
00430         if (id == -1)
00431         {
00432             pthread_mutex_unlock(&mutex);
00433             continue;
00434         }
00435
00436         patients[id].requestPending = 0;
00437
00438         /* Check appointment availability */
00439         if (activeAppointments < appointmentSlots)
00440         {
00441             patients[id].appointment = 1;
00442
00443             activeAppointments++;
00444             confirmedAppointments++;
00445
00446             printf("[Receptionist] %s (ID %d) - "
00447                    "Appointment CONFIRMED [%d/%d]\n",
00448                    patients[id].name,
00449                    patients[id].id,
00450                    activeAppointments,
00451                    appointmentSlots);
00452         }
00453         else
00454         {
00455             patients[id].appointment = 0;
00456
00457             printf("[Receptionist] %s (ID %d) - "
00458                    "No appointment slot. Walk-in allowed.\n",
00459                    patients[id].name,
00460                    patients[id].id);
00461         }
00462
00463         pthread_mutex_unlock(&mutex);
00464
00465         /* Reply to exact patient */
00466         sem_post(&patients[id].replySem);
00467     }
00468

```

```

00469     printf("[Receptionist] Closing.\n");
00470
00471     return NULL;
00472 }

```

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.11 showProjectInfo()

```
void showProjectInfo ()
```

Displays the contents of `project_info.txt`.

Reads and displays `project_info.txt`.

This function connects the external text documentation file with the C program.

Definition at line 365 of file [dentist.c](#).

```

00366 {
00367     FILE *file;
00368
00369     file = fopen("project_info.txt", "r");
00370
00371     if (file == NULL)
00372     {
00373         printf("\nproject_info.txt file not found.\n");
00374         return;
00375     }
00376
00377     char line[200];
00378
00379     printf("\n===== \n");
00380     printf("                PROJECT INFORMATION\n");
00381     printf("===== \n");
00382
00383     while (fgets(line, sizeof(line), file))
00384     {
00385         printf("%s", line);
00386     }
00387

```



```
00388     fclose(file);
00389
00390     printf("\n=====\\n");
00391 }
```

Here is the caller graph for this function:



4.1.3 Variable Documentation

4.1.3.1 activeAppointments

```
int activeAppointments = 0
```

Currently active appointment slots.

Definition at line [114](#) of file [dentist.c](#).

4.1.3.2 appointmentSlots

```
int appointmentSlots
```

Number of available appointment slots.

Definition at line [102](#) of file [dentist.c](#).

4.1.3.3 arrivalCounter

```
int arrivalCounter = 0
```

Counter used for FIFO / FCFS order.

Definition at line [117](#) of file [dentist.c](#).

4.1.3.4 clinicClosed

```
int clinicClosed = 0
```

Indicates whether the clinic is closed.

Definition at line [120](#) of file [dentist.c](#).

4.1.3.5 confirmedAppointments

```
int confirmedAppointments = 0
```

Total number of confirmed appointments.

Definition at line 111 of file [dentist.c](#).

4.1.3.6 doctorCapacity

```
int doctorCapacity
```

Maximum number of patients the doctor can treat.

Definition at line 99 of file [dentist.c](#).

4.1.3.7 mutex

```
pthread_mutex_t mutex
```

Mutex used to protect shared data.

This mutex protects critical sections where multiple threads access shared variables.

Definition at line 145 of file [dentist.c](#).

4.1.3.8 patientReady

```
sem_t patientReady
```

Semaphore used to notify the dentist when a patient enters the waiting room.

Definition at line 159 of file [dentist.c](#).

4.1.3.9 patients

```
struct patient* patients
```

Pointer to dynamically allocated patient array.

Definition at line 126 of file [dentist.c](#).

4.1.3.10 patientThreads

```
pthread_t* patientThreads
```

Pointer to dynamically allocated patient thread array.

Definition at line 132 of file [dentist.c](#).

4.1.3.11 requestReady

```
sem_t requestReady
```

Semaphore used to notify the receptionist when a patient requests an appointment.

Definition at line 152 of file [dentist.c](#).

4.1.3.12 totalPatients

```
int totalPatients
```

Total number of patients.

Definition at line 93 of file [dentist.c](#).

4.1.3.13 treatedPatients

```
int treatedPatients = 0
```

Number of patients already treated.

Definition at line 108 of file [dentist.c](#).

4.1.3.14 waitingPatients

```
int waitingPatients = 0
```

Current number of patients in waiting room.

Definition at line 105 of file [dentist.c](#).

4.1.3.15 waitingRoomChairs

```
int waitingRoomChairs
```

Number of chairs in the waiting room.

Definition at line 96 of file [dentist.c](#).

4.2 dentist.c

[Go to the documentation of this file.](#)

```

00001
00048
00049 #include <stdio.h>
00050 #include <stdlib.h>
00051 #include <string.h>
00052 #include <pthread.h>
00053 #include <semaphore.h>
00054 #include <unistd.h>
00055
00056
00064 struct patient
00065 {
00066     int id;
00067     char name[64];
00068
00069     int appointment;
00070     int willArrive;
00071
00072     int arrivalOrder;
00073     int waiting;
00074
00075     int treated;
00076     int noShow;
00077     int leftNoChair;
00078     int leftDoctorFull;
00079
00080     int requestPending;
00081
00082     sem_t replySem;
00083     sem_t dentistCallSem;
00084     sem_t treatmentDoneSem;
00085 };
00086
00087
00088 /* =====
00089     GLOBAL VARIABLES
00090     ===== */
00091
00093 int totalPatients;
00094
00096 int waitingRoomChairs;
00097
00099 int doctorCapacity;
00100
00102 int appointmentSlots;
00103
00105 int waitingPatients = 0;
00106
00108 int treatedPatients = 0;
00109
00111 int confirmedAppointments = 0;
00112
00114 int activeAppointments = 0;
00115
00117 int arrivalCounter = 0;
00118
00120 int clinicClosed = 0;
00121
00122
00126 struct patient *patients;
00127
00128
00132 pthread_t *patientThreads;
00133
00134
00135 /* =====
00136     MUTEX AND SEMAPHORES
00137     ===== */
00138
00145 pthread_mutex_t mutex;
00146
00147
00152 sem_t requestReady;
00153
00154
00159 sem_t patientReady;
00160
00161
00162 /* =====
00163     FUNCTION DECLARATIONS
00164     ===== */
00165

```

```

00171 void initializePatient(int i);
00172
00173
00179 void destroyPatient(int i);
00180
00181
00187 int findNextPatient();
00188
00189
00195 int findNextRequest();
00196
00197
00206 void *receptionist(void *arg);
00207
00208
00217 void *dentist(void *arg);
00218
00219
00229 void *patient(void *arg);
00230
00231
00235 void printPatientTable();
00236
00237
00241 void printStatistics();
00242
00243
00247 void showProjectInfo();
00248
00249
00250 /* =====
00251     INITIALIZE PATIENT
00252     ===== */
00253
00262 void initializePatient(int i)
00263 {
00264     patients[i].id = i + 1;
00265
00266     patients[i].appointment = 0;
00267     patients[i].willArrive = 1;
00268
00269     patients[i].arrivalOrder = 0;
00270     patients[i].waiting = 0;
00271
00272     patients[i].treated = 0;
00273     patients[i].noShow = 0;
00274     patients[i].leftNoChair = 0;
00275     patients[i].leftDoctorFull = 0;
00276
00277     patients[i].requestPending = 0;
00278
00279     sem_init(&patients[i].replySem, 0, 0);
00280     sem_init(&patients[i].dentistCallSem, 0, 0);
00281     sem_init(&patients[i].treatmentDoneSem, 0, 0);
00282 }
00283
00284
00285 /* =====
00286     DESTROY PATIENT
00287     ===== */
00288
00294 void destroyPatient(int i)
00295 {
00296     sem_destroy(&patients[i].replySem);
00297     sem_destroy(&patients[i].dentistCallSem);
00298     sem_destroy(&patients[i].treatmentDoneSem);
00299 }
00300
00301
00302 /* =====
00303     FIND NEXT PATIENT
00304     ===== */
00305
00313 int findNextPatient()
00314 {
00315     int selected = -1;
00316     int smallestOrder = 999999;
00317
00318     for (int i = 0; i < totalPatients; i++)
00319     {
00320         if (patients[i].waiting == 1 &&
00321             patients[i].arrivalOrder < smallestOrder)
00322         {
00323             smallestOrder = patients[i].arrivalOrder;
00324             selected = i;
00325         }
00326     }

```

```

00327
00328     return selected;
00329 }
00330
00331
00332 /* =====
00333 FIND NEXT REQUEST
00334 ===== */
00335
00341 int findNextRequest()
00342 {
00343     for (int i = 0; i < totalPatients; i++)
00344     {
00345         if (patients[i].requestPending == 1)
00346         {
00347             return i;
00348         }
00349     }
00350
00351     return -1;
00352 }
00353
00354
00355 /* =====
00356 SHOW PROJECT INFO
00357 ===== */
00358
00365 void showProjectInfo()
00366 {
00367     FILE *file;
00368
00369     file = fopen("project_info.txt", "r");
00370
00371     if (file == NULL)
00372     {
00373         printf("\nproject_info.txt file not found.\n");
00374         return;
00375     }
00376
00377     char line[200];
00378
00379     printf("\n=====\\n");
00380     printf("                PROJECT INFORMATION\\n");
00381     printf("=====\\n");
00382
00383     while (fgets(line, sizeof(line), file))
00384     {
00385         printf("%s", line);
00386     }
00387
00388     fclose(file);
00389
00390     printf("\n=====\\n");
00391 }
00392
00393
00394 /* =====
00395 RECEPTIONIST THREAD
00396 ===== */
00397
00407 void *receptionist(void *arg)
00408 {
00409     (void)arg;
00410
00411     printf("\n[Receptionist] Online.\\n");
00412
00413     while (1)
00414     {
00415         /* Wait for appointment request */
00416         sem_wait(&requestReady);
00417
00418         pthread_mutex_lock(&mutex);
00419
00420         /* Check clinic closing */
00421         if (clinicClosed == 1)
00422         {
00423             pthread_mutex_unlock(&mutex);
00424             break;
00425         }
00426
00427         /* Find request */
00428         int id = findNextRequest();
00429
00430         if (id == -1)
00431         {
00432             pthread_mutex_unlock(&mutex);
00433             continue;

```

```

00434     }
00435
00436     patients[id].requestPending = 0;
00437
00438     /* Check appointment availability */
00439     if (activeAppointments < appointmentSlots)
00440     {
00441         patients[id].appointment = 1;
00442
00443         activeAppointments++;
00444         confirmedAppointments++;
00445
00446         printf("[Receptionist] %s (ID %d) - "
00447                "Appointment CONFIRMED [%d/%d]\n",
00448                patients[id].name,
00449                patients[id].id,
00450                activeAppointments,
00451                appointmentSlots);
00452     }
00453     else
00454     {
00455         patients[id].appointment = 0;
00456
00457         printf("[Receptionist] %s (ID %d) - "
00458                "No appointment slot. Walk-in allowed.\n",
00459                patients[id].name,
00460                patients[id].id);
00461     }
00462
00463     pthread_mutex_unlock(&mutex);
00464
00465     /* Reply to exact patient */
00466     sem_post(&patients[id].replySem);
00467 }
00468
00469 printf("[Receptionist] Closing.\n");
00470
00471 return NULL;
00472 }
00473
00474 /* =====
00475  DENTIST THREAD
00476  ===== */
00477
00478 void *dentist(void *arg)
00479 {
00480     (void)arg;
00481
00482     printf("\n[Dr. Smith] Clinic is open.\n");
00483
00484     while (1)
00485     {
00486         /* Wait for patient */
00487         sem_wait(&patientReady);
00488
00489         pthread_mutex_lock(&mutex);
00490
00491         /* Close clinic when no patient remains */
00492         if (clinicClosed == 1 &&
00493             waitingPatients == 0)
00494         {
00495             pthread_mutex_unlock(&mutex);
00496             break;
00497         }
00498
00499         /* Doctor has treatment capacity */
00500         if (waitingPatients > 0 &&
00501             treatedPatients < doctorCapacity)
00502         {
00503             int id = findNextPatient();
00504
00505             if (id != -1)
00506             {
00507                 patients[id].waiting = 0;
00508                 waitingPatients--;
00509
00510                 printf("\n[Dr. Smith] Calling %s (ID %d)\n",
00511                        patients[id].name,
00512                        patients[id].id);
00513
00514                 printf("[Dr. Smith] FCFS Order: #%d\n",
00515                        patients[id].arrivalOrder);
00516
00517                 printf("[Dr. Smith] Waiting room: %d/%d\n",
00518                        waitingPatients,
00519                        waitingRoomChairs);

```

```

00530
00531     pthread_mutex_unlock(&mutex);
00532
00533     /* Call exact patient */
00534     sem_post(&patients[id].dentistCallSem);
00535
00536     printf("[Dr. Smith] Treating %s...\n",
00537           patients[id].name);
00538
00539     /* Treatment simulation */
00540     sleep(2);
00541
00542     printf("[Dr. Smith] Treatment finished for %s.\n",
00543           patients[id].name);
00544
00545     pthread_mutex_lock(&mutex);
00546
00547     treatedPatients++;
00548     patients[id].treated = 1;
00549
00550     /* Release appointment slot */
00551     if (patients[id].appointment == 1 &&
00552         activeAppointments > 0)
00553     {
00554         activeAppointments--;
00555     }
00556
00557     pthread_mutex_unlock(&mutex);
00558
00559     /* Notify patient */
00560     sem_post(&patients[id].treatmentDoneSem);
00561 }
00562 else
00563 {
00564     pthread_mutex_unlock(&mutex);
00565 }
00566 }
00567
00568 /* Doctor capacity reached */
00569 else if (waitingPatients > 0 &&
00570         treatedPatients >= doctorCapacity)
00571 {
00572     printf("\n[Dr. Smith] Daily treatment limit reached.\n");
00573     printf("[Dr. Smith] Remaining patients must leave.\n");
00574
00575     for (int i = 0; i < totalPatients; i++)
00576     {
00577         if (patients[i].waiting == 1)
00578         {
00579             patients[i].waiting = 0;
00580             patients[i].leftDoctorFull = 1;
00581
00582             waitingPatients--;
00583
00584             printf("[Dr. Smith] %s (ID %d) - "
00585                   "Left because doctor is full.\n",
00586                   patients[i].name,
00587                   patients[i].id);
00588
00589             sem_post(&patients[i].dentistCallSem);
00590         }
00591     }
00592
00593     pthread_mutex_unlock(&mutex);
00594 }
00595 else
00596 {
00597     pthread_mutex_unlock(&mutex);
00598 }
00599 }
00600
00601 printf("\n[Dr. Smith] Closing.\n");
00602
00603 return NULL;
00604 }
00605
00606
00607 /* =====
00608 PATIENT THREAD
00609 ===== */
00610
00625 void *patient(void *arg)
00626 {
00627     /*
00628      * Pointer to patient structure.
00629      */
00630     struct patient *p = (struct patient *)arg;

```



```

00631
00632
00633     /* -----
00634     REQUEST APPOINTMENT
00635     ----- */
00636
00637     printf("\n[Patient %d | %s] Requesting appointment.\n",
00638           p->id,
00639           p->name);
00640
00641     pthread_mutex_lock(&mutex);
00642
00643     p->requestPending = 1;
00644
00645     pthread_mutex_unlock(&mutex);
00646
00647     /* Notify receptionist */
00648     sem_post(&requestReady);
00649
00650     /* Wait for receptionist */
00651     sem_wait(&p->replySem);
00652
00653     printf("[Patient %d | %s] Appointment: ",
00654           p->id,
00655           p->name);
00656
00657     if (p->appointment == 1)
00658     {
00659         printf("CONFIRMED\n");
00660     }
00661     else
00662     {
00663         printf("NOT CONFIRMED / WALK-IN\n");
00664     }
00665
00666
00667     /* -----
00668     NO-SHOW
00669     ----- */
00670
00671     if (p->willArrive == 0)
00672     {
00673         pthread_mutex_lock(&mutex);
00674
00675         p->noShow = 1;
00676
00677         printf("\n[Patient %d | %s] NO-SHOW.\n",
00678               p->id,
00679               p->name);
00680
00681         /*
00682          * Release confirmed appointment slot.
00683          */
00684         if (p->appointment == 1)
00685         {
00686             if (activeAppointments > 0)
00687             {
00688                 activeAppointments--;
00689             }
00690
00691             printf("[Receptionist] %s's appointment slot "
00692                   "has been released.\n",
00693                   p->name);
00694
00695             printf("[Receptionist] Active appointments: "
00696                   "%d/%d\n",
00697                   activeAppointments,
00698                   appointmentsSlots);
00699         }
00700
00701         pthread_mutex_unlock(&mutex);
00702
00703         return NULL;
00704     }
00705
00706
00707     /* -----
00708     ARRIVE AT CLINIC
00709     ----- */
00710
00711     printf("[Patient %d | %s] Arrived at the clinic.\n",
00712           p->id,
00713           p->name);
00714
00715     pthread_mutex_lock(&mutex);
00716
00717

```

```

00718     /* Check doctor capacity */
00719
00720     if (treatedPatients >= doctorCapacity)
00721     {
00722         p->leftDoctorFull = 1;
00723
00724         printf("[Patient %d | %s] Doctor is full. "
00725             "Leaving clinic.\n",
00726             p->id,
00727             p->name);
00728
00729         pthread_mutex_unlock(&mutex);
00730
00731         return NULL;
00732     }
00733
00734
00735     /* -----
00736     ENTER WAITING ROOM
00737     ----- */
00738
00739     if (waitingPatients < waitingRoomChairs)
00740     {
00741         arrivalCounter++;
00742
00743         p->arrivalOrder = arrivalCounter;
00744         p->waiting = 1;
00745
00746         waitingPatients++;
00747
00748         printf("[Patient %d | %s] Sitting in waiting room.\n",
00749             p->id,
00750             p->name);
00751
00752         printf("[Patient %d | %s] Chair %d/%d | "
00753             "FCFS Order #%d\n",
00754             p->id,
00755             p->name,
00756             waitingPatients,
00757             waitingRoomChairs,
00758             p->arrivalOrder);
00759
00760         pthread_mutex_unlock(&mutex);
00761
00762         /* Notify dentist */
00763         sem_post(&patientReady);
00764
00765         /* Wait until dentist calls this patient */
00766         sem_wait(&p->dentistCallSem);
00767
00768         pthread_mutex_lock(&mutex);
00769
00770         if (p->leftDoctorFull == 1)
00771         {
00772             printf("[Patient %d | %s] Sent home because "
00773                 "doctor is full.\n",
00774                 p->id,
00775                 p->name);
00776
00777             pthread_mutex_unlock(&mutex);
00778
00779             return NULL;
00780         }
00781
00782         pthread_mutex_unlock(&mutex);
00783
00784         printf("[Patient %d | %s] Entering treatment room.\n",
00785             p->id,
00786             p->name);
00787
00788         /* Wait for treatment */
00789         sem_wait(&p->treatmentDoneSem);
00790
00791         printf("[Patient %d | %s] Treatment completed. "
00792             "Leaving clinic.\n",
00793             p->id,
00794             p->name);
00795     }
00796
00797
00798     /* -----
00799     WAITING ROOM FULL
00800     ----- */
00801
00802     else
00803     {
00804         p->leftNoChair = 1;

```

```

00805
00806     printf("[Patient %d | %s] Waiting room is FULL "
00807            "(%d/%d). Leaving clinic.\n",
00808            p->id,
00809            p->name,
00810            waitingPatients,
00811            waitingRoomChairs);
00812
00813     pthread_mutex_unlock(&mutex);
00814 }
00815
00816     return NULL;
00817 }
00818
00819
00820 /* =====
00821 PATIENT TABLE
00822 ===== */
00823
00827 void printPatientTable()
00828 {
00829     printf("\n");
00830
00831     printf("+-----+-----+-----+-----+-----+\n");
00832     printf("| ID   | Name           | Appointment   | Result         |\n");
00833     printf("+-----+-----+-----+-----+-----+\n");
00834
00835     for (int i = 0; i < totalPatients; i++)
00836     {
00837         char appointmentText[20];
00838         char resultText[30];
00839
00840         if (patients[i].appointment == 1)
00841         {
00842             strcpy(appointmentText, "Confirmed");
00843         }
00844         else
00845         {
00846             strcpy(appointmentText, "Not confirmed");
00847         }
00848
00849         if (patients[i].treated == 1)
00850         {
00851             strcpy(resultText, "Treated");
00852         }
00853         else if (patients[i].noShow == 1)
00854         {
00855             strcpy(resultText, "No-show");
00856         }
00857         else if (patients[i].leftNoChair == 1)
00858         {
00859             strcpy(resultText, "Room full");
00860         }
00861         else if (patients[i].leftDoctorFull == 1)
00862         {
00863             strcpy(resultText, "Doctor full");
00864         }
00865         else
00866         {
00867             strcpy(resultText, "Left");
00868         }
00869
00870         printf("| %-3d | %-16s | %-13s | %-16s |\n",
00871                patients[i].id,
00872                patients[i].name,
00873                appointmentText,
00874                resultText);
00875     }
00876
00877     printf("+-----+-----+-----+-----+-----+\n");
00878 }
00879
00880
00881 /* =====
00882 STATISTICS
00883 ===== */
00884
00888 void printStatistics()
00889 {
00890     int noShowCount = 0;
00891     int roomFullCount = 0;
00892     int doctorFullCount = 0;
00893
00894     for (int i = 0; i < totalPatients; i++)
00895     {
00896         if (patients[i].noShow == 1)
00897     
```

```

00898         noShowCount++;
00899     }
00900
00901     if (patients[i].leftNoChair == 1)
00902     {
00903         roomFullCount++;
00904     }
00905
00906     if (patients[i].leftDoctorFull == 1)
00907     {
00908         doctorFullCount++;
00909     }
00910 }
00911
00912 printf("\n");
00913
00914 printf("+-----+ \n");
00915 printf("|                               STATISTICS                               | \n");
00916 printf("+-----+ \n");
00917
00918 printf("| Total patients      : %-16d | \n",
00919        totalPatients);
00920
00921 printf("| Confirmed appointments: %-16d | \n",
00922        confirmedAppointments);
00923
00924 printf("| Patients treated    : %-16d | \n",
00925        treatedPatients);
00926
00927 printf("| No-show patients    : %-16d | \n",
00928        noShowCount);
00929
00930 printf("| Room-full patients  : %-16d | \n",
00931        roomFullCount);
00932
00933 printf("| Doctor-full patients : %-16d | \n",
00934        doctorFullCount);
00935
00936 printf("| Waiting room chairs  : %-16d | \n",
00937        waitingRoomChairs);
00938
00939 printf("| Doctor capacity     : %-16d | \n",
00940        doctorCapacity);
00941
00942 printf("| Appointment slots    : %-16d | \n",
00943        appointmentSlots);
00944
00945 printf("| Scheduling          : %-16s | \n",
00946        "FIFO / FCFS");
00947
00948 printf("+-----+ \n");
00949 }
00950
00951
00952 /* =====
00953 MAIN FUNCTION
00954 ===== */
00955
00965 int main()
00966 {
00967     int i;
00968
00969     printf("\n");
00970     printf("===== \n");
00971     printf("          DENTIST CLINIC SIMULATION \n");
00972     printf("          OS PROJECT \n");
00973     printf("===== \n");
00974
00975
00976     /* -----
00977     SHOW PROJECT INFORMATION
00978     ----- */
00979
00980     showProjectInfo();
00981
00982
00983     /* -----
00984     USER INPUT
00985     ----- */
00986
00987     printf("\nEnter number of patients: ");
00988     scanf("%d", &totalPatients);
00989
00990     if (totalPatients <= 0)
00991     {
00992         printf("Invalid number of patients. \n");
00993         return 1;

```

```

00994     }
00995
00996
00997     printf("Enter number of waiting room chairs: ");
00998     scanf("%d", &waitingRoomChairs);
00999
01000     if (waitingRoomChairs <= 0)
01001     {
01002         printf("Invalid number of chairs.\n");
01003         return 1;
01004     }
01005
01006
01007     printf("Enter doctor's maximum capacity: ");
01008     scanf("%d", &doctorCapacity);
01009
01010     if (doctorCapacity <= 0)
01011     {
01012         printf("Invalid doctor capacity.\n");
01013         return 1;
01014     }
01015
01016
01017     printf("Enter number of appointment slots: ");
01018     scanf("%d", &appointmentSlots);
01019
01020     if (appointmentSlots <= 0 ||
01021         appointmentSlots > totalPatients)
01022     {
01023         printf("Invalid appointment slot number.\n");
01024         return 1;
01025     }
01026
01027
01028     /* -----
01029     MEMORY ALLOCATION
01030     ----- */
01031
01032     patients =
01033         malloc(totalPatients * sizeof(struct patient));
01034
01035     patientThreads =
01036         malloc(totalPatients * sizeof(pthread_t));
01037
01038     if (patients == NULL ||
01039         patientThreads == NULL)
01040     {
01041         printf("Memory allocation failed.\n");
01042
01043         free(patients);
01044         free(patientThreads);
01045
01046         return 1;
01047     }
01048
01049
01050     /* -----
01051     PATIENT INPUT
01052     ----- */
01053
01054     printf("\n-----\n");
01055     printf("Enter patient information\n");
01056     printf("1 = Patient will come\n");
01057     printf("0 = Patient will not come (No-show)\n");
01058     printf("-----\n");
01059
01060     for (i = 0; i < totalPatients; i++)
01061     {
01062         initializePatient(i);
01063
01064         printf("\nPatient %d name: ",
01065             i + 1);
01066
01067         scanf("%63s",
01068             patients[i].name);
01069
01070         printf("Will %s come? (1=Yes, 0=No): ",
01071             patients[i].name);
01072
01073         scanf("%d",
01074             &patients[i].willArrive);
01075     }
01076
01077
01078     /* -----
01079     CLINIC SETTINGS
01080     ----- */

```

```

01081
01082     printf("\n=====\\n");
01083     printf("                CLINIC SETTINGS\\n");
01084     printf("=====\\n");
01085
01086     printf("Total patients      : %d\\n",
01087            totalPatients);
01088
01089     printf("Waiting room chairs : %d\\n",
01090            waitingRoomChairs);
01091
01092     printf("Doctor capacity      : %d\\n",
01093            doctorCapacity);
01094
01095     printf("Appointment slots   : %d\\n",
01096            appointmentSlots);
01097
01098     printf("Scheduling           : FIFO / FCFS\\n");
01099
01100     printf("=====\\n");
01101
01102
01103     /* -----
01104        INITIALIZE SYNCHRONIZATION
01105        ----- */
01106
01107     pthread_mutex_init(&mutex, NULL);
01108
01109     sem_init(&requestReady, 0, 0);
01110     sem_init(&patientReady, 0, 0);
01111
01112
01113     /* -----
01114        CREATE DENTIST THREAD
01115        ----- */
01116
01117     pthread_t dentistThread;
01118
01119     pthread_create(&dentistThread,
01120                   NULL,
01121                   dentist,
01122                   NULL);
01123
01124
01125     /* -----
01126        CREATE RECEPTIONIST THREAD
01127        ----- */
01128
01129     pthread_t receptionistThread;
01130
01131     pthread_create(&receptionistThread,
01132                   NULL,
01133                   receptionist,
01134                   NULL);
01135
01136
01137     printf("\n=====\\n");
01138     printf("                SIMULATION STARTED\\n");
01139     printf("=====\\n");
01140
01141
01142     /* -----
01143        CREATE PATIENT THREADS
01144        ----- */
01145
01146     for (i = 0; i < totalPatients; i++)
01147     {
01148         pthread_create(&patientThreads[i],
01149                       NULL,
01150                       patient,
01151                       &patients[i]);
01152
01153         /*
01154          * Small delay between patient arrivals.
01155          */
01156         usleep(300000);
01157     }
01158
01159
01160     /* -----
01161        WAIT FOR PATIENT THREADS
01162        ----- */
01163
01164     for (i = 0; i < totalPatients; i++)
01165     {
01166         pthread_join(patientThreads[i],
01167                      NULL);

```

```

01168     }
01169
01170
01171     /* -----
01172        CLOSE CLINIC
01173        ----- */
01174
01175     pthread_mutex_lock(&mutex);
01176
01177     clinicClosed = 1;
01178
01179     pthread_mutex_unlock(&mutex);
01180
01181
01182     /* Wake dentist and receptionist */
01183     sem_post(&patientReady);
01184     sem_post(&requestReady);
01185
01186
01187     /* Wait for dentist */
01188     pthread_join(dentistThread,
01189                 NULL);
01190
01191
01192     /* Wait for receptionist */
01193     pthread_join(receptionistThread,
01194                 NULL);
01195
01196
01197     /* -----
01198        FINAL RESULT
01199        ----- */
01200
01201     printf("\n=====\\n");
01202     printf("                CLINIC CLOSED\\n");
01203     printf("=====\\n");
01204
01205     printPatientTable();
01206
01207     printStatistics();
01208
01209
01210     /* -----
01211        CLEANUP
01212        ----- */
01213
01214     pthread_mutex_destroy(&mutex);
01215
01216     sem_destroy(&requestReady);
01217     sem_destroy(&patientReady);
01218
01219     for (i = 0; i < totalPatients; i++)
01220     {
01221         destroyPatient(i);
01222     }
01223
01224     free(patients);
01225     free(patientThreads);
01226
01227     printf("\nAll threads terminated successfully.\\n");
01228     printf("Program finished.\\n\\n");
01229
01230     return 0;
01231 }
01232

```


Index

- activeAppointments
 - dentist.c, [29](#)
- appointment
 - patient, [5](#)
- appointmentSlots
 - dentist.c, [29](#)
- arrivalCounter
 - dentist.c, [29](#)
- arrivalOrder
 - patient, [5](#)

- clinicClosed
 - dentist.c, [29](#)
- confirmedAppointments
 - dentist.c, [29](#)

- dentist
 - dentist.c, [12](#)
- dentist.c, [9](#)
 - activeAppointments, [29](#)
 - appointmentSlots, [29](#)
 - arrivalCounter, [29](#)
 - clinicClosed, [29](#)
 - confirmedAppointments, [29](#)
 - dentist, [12](#)
 - destroyPatient, [14](#)
 - doctorCapacity, [30](#)
 - findNextPatient, [14](#)
 - findNextRequest, [15](#)
 - initializePatient, [16](#)
 - main, [17](#)
 - mutex, [30](#)
 - patient, [21](#)
 - patientReady, [30](#)
 - patients, [30](#)
 - patientThreads, [30](#)
 - printPatientTable, [24](#)
 - printStatistics, [25](#)
 - receptionist, [26](#)
 - requestReady, [30](#)
 - showProjectInfo, [28](#)
 - totalPatients, [31](#)
 - treatedPatients, [31](#)
 - waitingPatients, [31](#)
 - waitingRoomChairs, [31](#)
- dentistCallSem
 - patient, [6](#)
- destroyPatient
 - dentist.c, [14](#)
- doctorCapacity
 - dentist.c, [30](#)
- findNextPatient
 - dentist.c, [14](#)
- findNextRequest
 - dentist.c, [15](#)

- id
 - patient, [6](#)
- initializePatient
 - dentist.c, [16](#)

- leftDoctorFull
 - patient, [6](#)
- leftNoChair
 - patient, [6](#)

- main
 - dentist.c, [17](#)
- mutex
 - dentist.c, [30](#)

- name
 - patient, [6](#)
- noShow
 - patient, [6](#)

- patient, [5](#)
 - appointment, [5](#)
 - arrivalOrder, [5](#)
 - dentist.c, [21](#)
 - dentistCallSem, [6](#)
 - id, [6](#)
 - leftDoctorFull, [6](#)
 - leftNoChair, [6](#)
 - name, [6](#)
 - noShow, [6](#)
 - replySem, [7](#)
 - requestPending, [7](#)
 - treated, [7](#)
 - treatmentDoneSem, [7](#)
 - waiting, [7](#)
 - willArrive, [7](#)
- patientReady
 - dentist.c, [30](#)
- patients
 - dentist.c, [30](#)
- patientThreads
 - dentist.c, [30](#)
- printPatientTable
 - dentist.c, [24](#)

printStatistics
dentist.c, [25](#)

receptionist
dentist.c, [26](#)

replySem
patient, [7](#)

requestPending
patient, [7](#)

requestReady
dentist.c, [30](#)

showProjectInfo
dentist.c, [28](#)

totalPatients
dentist.c, [31](#)

treated
patient, [7](#)

treatedPatients
dentist.c, [31](#)

treatmentDoneSem
patient, [7](#)

waiting
patient, [7](#)

waitingPatients
dentist.c, [31](#)

waitingRoomChairs
dentist.c, [31](#)

willArrive
patient, [7](#)